

# Sesión 6: Introducción a Docker

Contenedores, Imágenes y Orquestación Básica

**Prof. Alejandro Peñaranda Agudelo**

Universidad ICESI — Período 202620

### 1 **Introducción y Contenedores**

2 Imágenes y Dockerfiles

3 Redes y Puertos

4 Volúmenes y Compose

01

# Introducción y Contenedores

Entendiendo la virtualización a nivel de sistema operativo

- **El caos clásico:** Correr múltiples aplicaciones con distintas versiones de librerías y dependencias sobre el mismo servidor físico.
- **La brecha “en mi máquina funciona”:** Diferencias de sistemas operativos, variables y dependencias entre Dev, Test y Producción.
- **La analogía del contenedor:** Al igual que los contenedores de carga estandarizaron el transporte de mercancías (sin importar si es café o electrónica), Docker estandariza el empaquetado de software.

### El Problema

¿Cómo garantizar que un software se ejecute de forma idéntica en cualquier entorno físico o en la nube sin importar sus dependencias?

## Máquinas Virtuales (VM)

- **Hipervisor:** Virtualizan el hardware completo.
- **SO Huésped:** Cada VM corre un SO completo (giga-bytes de espacio, minutos en arrancar).
- **Aislamiento:** Fuerte aislamiento de hardware, pero alto consumo de recursos.

## Contenedores (Docker)

- **Motor de Docker:** Virtualiza a nivel de SO (comparte el Kernel del host).
- **Ligero:** Solo empaqueta la app y sus binarios de usuario (mega-bytes, milisegundos en arrancar).
- **Eficiencia:** Alta densidad de servicios en la misma máquina física.

Docker no es magia; aprovecha características nativas del Kernel de Linux:

- **Namespaces (Aislamiento):** Aíslan lo que ve el contenedor: procesos (PID), red (NET), sistemas de archivos (MNT) y usuarios (USER).
- **Control Groups / Cgroups (Límites):** Limitan los recursos físicos que puede consumir un contenedor (ej. máximo 1 GB de RAM y 0.5 CPU).
- **Chroot:** Cambia la raíz del sistema de archivos del proceso al contenedor.

### Aislamiento

Los namespaces y cgroups permiten que un contenedor actúe como un entorno aislado e independiente compartiendo la velocidad del Kernel anfitrión.

1 Introducción y Contenedores

**2 Imágenes y Dockerfiles**

3 Redes y Puertos

4 Volúmenes y Compose

02

# Imágenes y Dockerfiles

Empaquetamiento y optimización de software en capas

- **Plantilla de lectura:** Paquete inmutable que contiene el código, runtime, librerías y configuraciones.
- **Sistema de archivos en capas (UnionFS):** Cada instrucción del archivo de receta crea una nueva capa de lectura.
- **Capa de escritura (Container Layer):** Al ejecutar una imagen, Docker añade una delgada capa temporal de escritura en la parte superior.
- **Copy-on-Write (CoW):** Optimiza el almacenamiento compartiendo las capas base inmutables entre múltiples contenedores.

### Almacenamiento

Las capas inmutables permiten que múltiples contenedores compartan casi todo su sistema de archivos base, ocupando espacio en disco insignificante.

- **Imagen Base (FROM):** Define el punto de partida (ej: FROM node:18-alpine).
- **Estructura (WORKDIR y COPY):** Configura el directorio interno de ejecución y copia los archivos del proyecto.
- **Comandos (RUN vs. CMD):** RUN ejecuta instrucciones durante la compilación (crea capa); CMD define el proceso inicial al iniciar el contenedor.
- **Caché de Capas:** Colocar las directivas estables (como instalar dependencias) antes de copiar el código cambiante optimiza los tiempos de compilación.

### Optimización

Si una línea y sus archivos asociados no cambian, Docker reutiliza la capa guardada en caché, acelerando drásticamente las compilaciones del pipeline.

- **El problema:** El compilador (ej. JDK, Maven) es pesado y no se necesita en producción; solo se requiere el ejecutable final (ej. archivo jar).
- **Multi-stage:** Permite usar múltiples instrucciones FROM en el mismo Dockerfile para crear etapas temporales.
- **Etapas de compilación:** Usa una imagen pesada con compiladores y dependencias para generar el binario.
- **Etapas final de ejecución:** Copia únicamente el binario compilado hacia una imagen limpia y liviana (ej: alpine o distroless).

### Optimización

Las etapas múltiples permiten reducir el peso de las imágenes finales de producción hasta en un 90 %, disminuyendo la superficie de vulnerabilidad.

1 Introducción y Contenedores

2 Imágenes y Dockerfiles

**3 Redes y Puertos**

4 Volúmenes y Compose

# 03

## Redes y Puertos

Configurando la conectividad e interconexión de contenedores

- **Aislamiento de red:** Por defecto, los puertos expuestos dentro de un contenedor no son accesibles desde el host ni desde la red externa.
- **La Bandera -p:** Mapea un puerto del host anfitrión a un puerto interno del contenedor (-p puerto\_host:puerto\_contenedor).
- **Traducción de Red (NAT):** Docker gestiona reglas internas a nivel del kernel para enrutar el tráfico entrante.

### Consola

```
docker run -d -p  
8080:3000 mi-app-web  
Acceso desde el host:  
http://localhost:8080  
hacia el contenedor en  
puerto 3000.
```

### Bridge (Por Defecto)

Crea una red virtual privada local en el host.

- Los contenedores obtienen una IP en la subred interna de Docker.
- Aislamiento estándar y redirección a través de puertos.

### Host y None

- **Host:** Elimina el aislamiento de red. El contenedor usa la red y puertos del host directamente (más rápido, menos seguro).
- **None:** Aislamiento completo. Sin interfaz de red externa, ideal para tareas de cálculo aisladas.

- **El problema de la red por defecto:** En la red bridge por defecto, los contenedores no pueden resolverse entre sí por su nombre.
- **Red bridge personalizada:** Crear una red propia permite la resolución automática de nombres DNS embebida.
- **Conexión por nombre:** Un contenedor llamado db-service es accesible por otro contenedor en la misma red usando el hostname `http://db-service:27017` sin importar el cambio de IPs.

### Red de Equipos

```
docker network create  
mi-red-devops  
docker run -d  
--name db --network  
mi-red-devops mongo  
docker run -d  
--name web --network  
mi-red-devops mi-web
```

1 Introducción y Contenedores

2 Imágenes y Dockerfiles

3 Redes y Puertos

**4 Volúmenes y Compose**

# 04

## Volúmenes y Compose

Persistencia de datos y orquestación multi-contenedor local

### Volúmenes de Docker

Directorios gestionados por Docker en una zona reservada del host (`/var/lib/docker/volumes`).

- **Seguro:** Aislado de modificaciones accidentales del host.
- **Rendimiento:** Óptimo para base de datos persistentes.

### Bind Mounts

Mapeo directo de un directorio arbitrario del host al contenedor.

- **Ideal para desarrollo:** Permite cambios en tiempo real en el código del host que se reflejan al instante en el contenedor.
- **Portabilidad:** Depende de la estructura de archivos del host.

- **El reto multi-contenedor:** Levantar una arquitectura (Frontend, Backend, DB) requiere teclear comandos largos y coordinar el orden de arranque manualmente.
- **Docker Compose:** Herramienta para definir y correr aplicaciones multi-contenedor mediante un archivo YAML declarativo (`docker-compose.yml`).
- **Unificación:** Define servicios, volúmenes y redes en un único archivo versionado.

### Comando Único

Con un solo comando declarativo: `docker compose up -d`, Docker levanta toda la red, volúmenes y contenedores en el orden correcto.

Estructura YAML típica para el taller de la calculadora distribuida:

- `services:` — Define los contenedores a crear.
- `backend:` — Servicio de cálculo en puerto interno 5000.
- `frontend:` — Servicio cliente en puerto 8080, mapea al host 8080:8080. Depende del backend (`depends_on`).
- `database:` — Servicio base de datos que usa un volumen para persistencia.
- `volumes:` y `networks:` — Declara recursos compartidos automáticamente.

### Declarativo

El archivo de configuración codifica la topología de red y el enrutamiento de la solución, reemplazando la documentación y los scripts manuales de red.

- **Consistencia absoluta:** Garantiza que el software se comporte exactamente igual desde la máquina de desarrollo hasta producción.
- **Inmutable y versionado:** La infraestructura de ejecución se define en código (Dockerfile y Compose) dentro del repositorio Git.
- **Habilitador de CI/CD:** Permite levantar entornos de prueba efímeros en segundos para validar Pull Requests.

### Mantra

La infraestructura de despliegue ya no es tarea oculta de Ops; ahora se define en código compartido y versionado junto a la aplicación.

- 1. Si los contenedores comparten el Kernel del host, ¿se puede correr un contenedor nativo de Windows dentro de un host Linux?
- 2. ¿Cuál es el peligro de seguridad de correr todos los procesos de tu aplicación como usuario root dentro de un contenedor?
- 3. Si borras un contenedor que no tiene volúmenes asociados, ¿qué ocurre con los datos generados durante su ejecución?
- 4. ¿Por qué es un antipatrón almacenar contraseñas de bases de datos hardcodeadas en la directiva ENV de un Dockerfile?

### Pregunta 5

¿Cómo facilita Docker Compose el cumplimiento de la filosofía de equipos pequeños e independientes en microservicios?